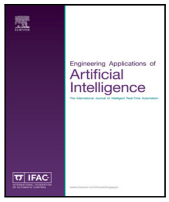
Engineering Applications of Artificial Intelligence 118 (2023) 105688



Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai



A reinforcement learning based computational intelligence approach for binary optimization problems: The case of the set-union knapsack problem



Fehmi Burcin Ozsoydan [a,*], İlker Gölcük [b]

[a] Department of Industrial Engineering, Faculty of Engineering, Dokuz Eylül University, İzmir 35397, Türkiye
[b] Department of Industrial Engineering, İzmir Bakırçay University, İzmir 35665, Türkiye

ARTICLE INFO

Keywords:
Machine learning
Q-learning
Genetic algorithm
Particle swarm optimization
Binary optimization
Knapsack problem

ABSTRACT

As today's one of the hottest topics, machine learning brings about opportunities in various research areas. Moreover, computational intelligence and metaheuristics open up new strategies, which are shown to be efficient in solving optimization problems. However, studies bringing such remarkable approaches together are still lacking. In this context, the present paper introduces a Q-learning reinforcement learning strategy for binary optimization problems. The developed algorithm works as a reinforcement and recommendation system that evaluates the used algorithms, assigns rewards, promotes or demotes them. Thus, it invokes more promising optimizers more frequently. The proposed Q-learning algorithm uses Particle Swarm Optimization (PSO), Genetic Algorithm and a hybrid of these algorithms, namely, genetic-based PSO (gbPSO) as optimizers. Therefore, it is aimed to avoid local optima by using various optimizers and gathering additional statistical data. Secondarily, all optimizers are further enhanced by adopting an initial solution generation technique and triggered random immigrants mechanism to preserve swarm diversity. In addition to these procedures, a mutation procedure that decreases the diversity is adopted. Thus, more intensified search is encouraged towards the end of search. Moreover, while PSO requires for transfer functions in order to perform in binary spaces, the adopted and further improved gbPSO does not necessarily need such auxiliary procedures. Finally, the performances of all used algorithms are analysed on a recently caught on binary problem, namely, the set-union knapsack problem, which has a wide range of real-life applications. As demonstrated by the comprehensive experimental study and appropriate statistical tests, promising improvements are achieved.

1. Introduction

Machine learning (Alpaydin, 2021) can be considered as one of the hottest topics lately. It brings about outstanding opportunities in various research areas including supervised learning, unsupervised learning, reinforcement learning and optimization. Such learning methods can be employed in regression analysis, image processing, classification, pattern recognition, prediction, robotics and signal processing (Khan et al., 2001; Uslu and Celik, 2018; Maleki et al., 2019; Yilmaz et al., 2019; Farzaneh-Gord et al., 2020; Turkoglu and Kaya, 2020). In parallel with such developments in artificial intelligence research, computational intelligence algorithms including swarm-based optimization techniques and metaheuristics are shown to be efficient in solving optimization problems.

As reported by a number of reported surveys (Gogna and Tayal, 2013; Boussaïd et al., 2013; Hussain et al., 2019), there are various swarm intelligence-based metaheuristics. One can put forward that Genetic Algorithm (GA) (Holland, 1975) is one of the well-known algorithms in this domain since it has numerous applications in both artificial intelligence and computational intelligence research fields, which are indeed intertwined topics. Since GAs simulate the evolutionary mechanisms of Darwinian Theory such as crossover, mutation and natural selection, they are also classified in the evolutionary algorithms category, which is also another branch of artificial intelligence. Those well-known operators of GAs often inspire other algorithms as well. As another well-regarded algorithm, Differential Evolution Algorithm (DE) (Storn and Price, 1995) for instance, performs differentiation operations between solution strings that are followed by crossover and mutation, adopted from GAs. Another well-regarded algorithm, which is employed in numerous studies, Particle Swarm Optimization (PSO) (Eberhart and Kennedy, 1995; Kennedy and Eberhart, 1995) simulates the behaviours of bird or fish flocks. Such well-known computational intelligence algorithms are shown to be capable of achieving remarkable results in numerous studies. However, studies bringing the capabilities of such promising approaches together with the machine learning methods are still lacking in the related literature.

In this context, this study is motivated by this issue and presents a Q-learning algorithm that works as a reinforcement and algorithm

* Corresponding author.
E-mail addresses: burcin.ozsoydan@deu.edu.tr (F.B. Ozsoydan), ilker.golcuk@bakircay.edu.tr (İ. Gölcük).

https://doi.org/10.1016/j.engappai.2022.105688
Received 10 March 2022; Received in revised form 25 August 2022; Accepted 27 November 2022
Available online 13 December 2022
0952-1976/© 2022 Elsevier Ltd. All rights reserved.

Table 1
SUKP instances.

#	Instance label	#	Instance label	#	Instance label
Fi1	100_85_0.10_0.75	Si1	100_100_0.10_0.75	Ti1	85_100_0.10_0.75
Fi2	100_85_0.15_0.85	Si2	100_100_0.15_0.85	Ti2	85_100_0.15_0.85
Fi3	200_185_0.10_0.75	Si3	200_200_0.10_0.75	Ti3	185_200_0.10_0.75
Fi4	200_185_0.15_0.85	Si4	200_200_0.15_0.85	Ti4	185_200_0.15_0.85
Fi5	300_285_0.10_0.75	Si5	300_300_0.10_0.75	Ti5	285_300_0.10_0.75
Fi6	300_285_0.15_0.85	Si6	300_300_0.15_0.85	Ti6	285_300_0.15_0.85
Fi7	400_385_0.10_0.75	Si7	400_400_0.10_0.75	Ti7	385_400_0.10_0.75
Fi8	400_385_0.15_0.85	Si8	400_400_0.15_0.85	Ti8	385_400_0.15_0.85
Fi9	500_485_0.10_0.75	Si9	500_500_0.10_0.75	Ti9	485_500_0.10_0.75
Fi10	500_485_0.15_0.85	Si10	500_500_0.15_0.85	Ti10	485_500_0.15_0.85

Table 2
Computational results of the first type of instances.

#	Results	A-SUKP	GA	BABC	ABC_{bin}	binDE	gPSO	GA^*	PSO	QL	gbPSO
Fi1	*best*	12459	13044	13251	13044	13044	**13283**	13167	13044	13251	**13283**
	mean	12459	12956.4	13028.5	12818.5	12991	13050.53	12938.55	13025.13	13049.88	13049.42
	std. dev.	0.00	130.66	92.63	153.06	75.95	37.41	135.50	38.08	27.65	27.96
	worst	–	–	–	–	–	13044	12539	12908	13044	13044
Fi2	*best*	11119	12066	12238	12238	12274	12274	**12479**	12210	**12479**	12348
	mean	11119	11546	12155	12049.3	12123.9	12084.82	12093.26	12023.50	12101.41	12108.39
	std. dev.	0.00	214.94	53.29	96.11	67.61	95.38	97.05	87.62	87.80	80.14
	worst	–	–	–	–	–	11631	11715	11655	11912	11985
Fi3	*best*	11292	13064	13241	12946	13241	**13405**	13137	13043	**13405**	**13405**
	mean	11292	12492.5	13064.4	11861.5	12940.7	13286.56	12468.53	12139.54	13075.43	13308.94
	std. dev.	0.00	320.03	99.57	324.65	205.70	93.18	349.45	326.85	209.89	95.09
	worst	–	–	–	–	–	12862	11649	11396	12438	12818
Fi4	*best*	12262	13671	13829	13671	13671	14044	13671	13220	14044	**14215**
	mean	12262	12802.9	13359.2	12537	13110	13492.60	12984.63	12759.72	13309.32	13595.80
	std. dev.	0.00	291.66	234.99	289.53	269.69	328.72	253.84	207.12	294.52	365.53
	worst	–	–	–	–	–	12785	12416	12237	12727	12934
Fi5	*best*	8941	10553	10428	9751	10420	**11335**	10235	9902	10952	**11335**
	mean	8941	9980.87	9994.76	9339.3	9899.24	10669.51	9644.61	9614.90	10281.50	10719.21
	std. dev.	0.00	142.97	154.03	158.15	153.18	227.85	191.29	135.12	255.44	192.55
	worst	–	–	–	–	–	9900	9266	9366	9774	10116
Fi6	*best*	9432	11016	12012	10913	11661	**12245**	11346	10890	**12245**	**12245**
	mean	9432	10349.8	10902.9	9957.85	10499.4	11607.10	10356.40	9966.94	11096.13	11660.46
	std. dev.	0.00	215.13	449.45	276.90	403.95	477.80	363.74	236.47	576.70	481.10
	worst	–	–	–	–	–	10536	9599	9492	10049	10438
Fi7	*best*	9076	10083	10766	9674	10576	**11484**	10322	9577	11242	**11484**
	mean	9076	9641.85	10065.2	9187.76	9681.46	10915.87	9529.54	9212.33	10431.49	10845.05
	std. dev.	0.00	168.94	241.45	167.08	275.05	367.75	240.38	153.60	395.26	376.94
	worst	–	–	–	–	–	10158	9069	8964	9492	9895
Fi8	*best*	8514	9831	9649	8978	9649	**10710**	9519	9107	10419	10526
	mean	8514	9326.77	9135.98	8539.95	9020.87	9864.55	8808.69	8773.72	9580.06	9846.91
	std. dev.	0.00	192.20	151.90	161.83	150.99	315.38	190.65	123.98	338.07	354.96
	worst	–	–	–	–	–	9173	8423	8507	8843	8806
Fi9	*best*	9864	11031	10784	10340	10586	11722	10483	10483	11414	**11771**
	mean	9864	10567.9	10452.2	9910.32	10363.8	11184.51	10184.74	10179.86	10837.59	11217.69
	std. dev.	0.00	123.15	114.35	120.82	93.39	322.98	130.56	96.96	253.40	354.47
	worst	–	–	–	–	–	10614	9912	9949	10332	10577
Fi10	*best*	8299	9472	9090	8759	9191	**10022**	8926	8927	9729	**10022**
	mean	8299	8692.67	8857.89	8365.04	8783.99	9299.56	8529.32	8408.39	8993.48	9222.81
	std. dev.	0.00	180.12	94.55	114.10	131.05	277.62	148.05	118.55	216.57	241.45S
	worst	–	–	–	–	–	8556	8251	8190	8488	8786

recommendation system, which considers the performances of the used optimizers, assigns rewards, promotes or demotes them. The proposed Q-learning algorithm employs PSO, GA and a hybrid of these algorithms, namely, genetic-based PSO (gbPSO) (Ozsoydan and Baykasoglu, 2019) as metaheuristic optimizers. Therefore, by using a variety of different optimizers, it is aimed to avoid possible local optima and secondarily, it is aimed to gather additional statistical data to learn and to claim the more promising algorithms. It is also clear that the Q-learning algorithm does not directly work on problem space; it works on the algorithm space. It only chooses optimizers, which work on problem space. Throughout generations, it learns about the performances of the algorithms and attempts to invoke them more frequently while giving occasional chances to more naïve algorithms.

Secondarily, all optimizers are further enhanced by adopting an improved initial solution generation technique and triggered random immigrants (TRIM) mechanism that is devised to preserve swarm diversity to avoid possible local optima. Additionally, gbPSO is extended so as to be able to use the information of other individuals in population. The adopted and further improved gbPSO does not necessarily require for transfer functions in order to perform in binary spaces. Finally, a mutation procedure that decreases the introduced diversity is adopted. Thus, while exploration is promoted at initialization stage, more intensified search is encouraged towards the end of search. The performances of all used algorithms are analysed on a recently caught on binary optimization problem, namely, the set-union knapsack problem (SUKP), which has a wide range of real-life applications (He et al.,

Table 3
Computational results of the second type of instances.

#	Results	A-SUKP	GA	BABC	ABC_{bin}	binDE	gPSO	GA*	PSO	QL	gbPSO
Si1	*best*	13634	**14044**	13860	13860	13814	**14044**	13849	13849	**14044**	**14044**
	mean	13634	13806	13734.9	13547.2	13675.9	13854.71	13665.98	13704.26	13851.28	13865.98
	std. dev.	0.00	144.91	70.76	119.11	119.53	96.23	96.70	62.57	78.86	84.19
	worst	–	–	–	–	–	13664	13355	13531	13664	13664
Si2	*best*	11325	13145	**13508**	13498	13407	**13508**	**13508**	13407	**13508**	**13508**
	mean	11325	12234.8	13352.4	13103.1	13212.8	13347.58	13176.40	13271.17	13343.37	13375.41
	std. dev.	0.00	388.66	155.14	343.46	287.45	194.34	264.82	136.51	176.81	154.28
	worst	–	–	–	–	–	12613	12433	12836	12809	12860
Si3	*best*	10328	11656	11846	11191	11535	**12522**	11648	11450	12196	**12522**
	mean	10328	10888.7	11194.3	10424.1	10969.4	11898.73	10849.64	10612.58	11408.12	11995.98
	std. dev.	0.00	237.85	249.58	197.88	302.52	391.83	305.54	188.99	387.73	320.78
	worst	–	–	–	–	–	11048	10233	10329	10626	11150
Si4	*best*	9784	11792	11521	11287	11469	**12317**	11365	11404	11911	**12317**
	mean	9784	10827.5	10945	10345.9	10717.1	11584.64	10779.49	10655.96	11458.29	11772.21
	std. dev.	0.00	334.43	255.14	273.47	341.08	275.32	323.81	285.92	313.89	200.78
	worst	–	–	–	–	–	10510	10152	10223	10404	11184
Si5	*best*	10208	12055	12186	11494	12304	**12695**	12040	12052	12644	**12695**
	mean	10208	11755.1	11945.8	10922.3	11864.4	12411.27	11468.21	11363.44	12019.86	12404.31
	std. dev.	0.00	144.45	127.80	182.63	160.42	225.80	274.09	193.94	222.38	250.50
	worst	–	–	–	–	–	11786	10934	11057	11553	11865
Si6	*best*	9183	10666	10382	9633	10382	**11425**	10467	9592	11282	**11425**
	mean	9183	10099.2	9859.69	9186.87	9710.37	10568.41	9476.72	9344.28	10242.21	10555.03
	std. dev.	0.00	337.42	177.02	147.78	208.48	327.48	216.10	116.83	347.52	326.82
	worst	–	–	–	–	–	9648	9029	9056	9528	9700
Si7	*best*	9751	10570	10626	10160	10462	**11531**	10372	9970	11158	11432
	mean	9751	10112.4	10101.1	9549.04	9975.8	10958.96	9813.67	9718.50	10465.86	11012.87
	std. dev.	0.00	157.89	196.99	141.27	185.57	274.90	207.62	99.46	246.03	236.15
	worst	–	–	–	–	–	10205	9458	9497	9844	10562
Si8	*best*	8497	9235	9541	9033	9388	**10927**	9325	8900	10520	**10927**
	mean	8497	8793.76	9032.95	8365.62	8768.42	9845.17	8624.81	8406.31	9283.62	9742.38
	std. dev.	0.00	169.52	194.18	153.40	212.24	358.91	221.91	144.69	358.36	403.36
	worst	–	–	–	–	–	9033	8264	8148	8619	8922
Si9	*best*	9615	10460	10755	10071	10546	10888	10412	10273	10888	**10991**
	mean	9615	10185.4	10328.5	9738.17	10227.7	10681.46	9997.09	9900.67	10435.59	10662.23
	std. dev.	0.00	114.19	91.615	111.63	103.32	125.36	137.39	83.42	178.33	168.62
	worst	–	–	–	–	–	10222	9690	9740	9918	10189
Si10	*best*	7883	9496	9318	9262	9312	**10194**	9546	9203	9938	**10194**
	mean	7883	8882.88	9180.74	8617.91	9096.13	9703.62	8879.65	8727.18	9393.54	9637.06
	std. dev.	0.00	158.21	84.91	141.32	145.45	252.84	178.46	165.10	254.37	255.63
	worst	–	–	–	–	–	8892	8540	8441	8755	9071

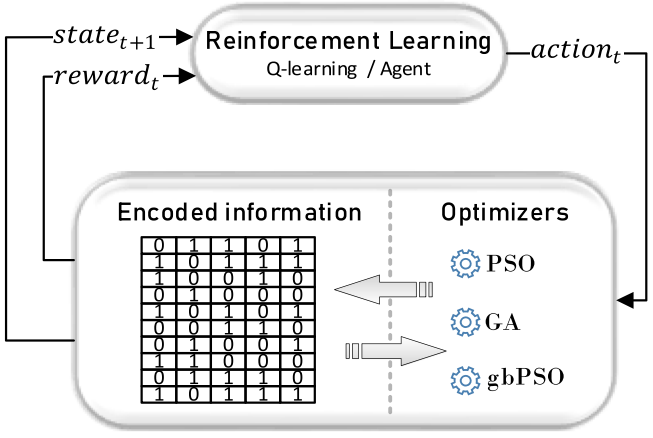


Fig. 1. An illustration for the used Q-learning algorithm.

2018). Conducted statistical tests and comprehensive study, point out the efficiency of the proposed approach.

The rest of the paper is organized as follows: SUKP and its related literature are introduced in Section 2, while the details of all used and developed algorithms are given in Section 3. Finally, experimental results and concluding remarks are presented in Section 4 and Section 5, respectively.

2. The set-union knapsack problem

Binary optimization problems are of interest to researchers due to their variety of real-life applications. One of the commonly-known binary optimization problem is the 0–1 Knapsack Problem (Kellerer et al., 2004), which is closely related to numerous optimization problems such as project funds selection, cargo loading, budget management, cutting stock, allocating processors in distributed computer systems, etc. SUKP can be regarded as an extension of the 0–1 Knapsack Problem (KP). In the 0–1 KP, items have profits and weights and knapsack has a limited capacity. Similarly, in SUKP, each item is associated with profits, knapsack has a limited capacity and the aim is to maximize the total profit. However, distinctly from 0–1 KP, items do not have weights here. Elements, which are associated with items, have weights in SUKP. Thus, total consumed capacity is calculated by sum of the weights of all elements in the union set, which includes the elements of the assigned items. SUKP can be formulated as given in the following (Arulselvan, 2014):

$$maximize\ P(A) = \sum_{i \in A} p_i \quad s.t. \tag{1}$$

Table 4
Computational results of the third type of instances.

Instance	Results	A-SUKP	GA	BABC	ABC_{bin}	binDE	gPSO	GA^*	PSO	QL	gbPSO
Ti1	*best*	10231	11454	11664	11206	11352	**12045**	11530	11576	11619	**12045**
	mean	10231	11092.7	11182.7	10879.5	11075	11486.95	11028.38	11027.45	11371.96	11525.48
	std. dev.	0.00	171.22	183.57	163.62	119.42	137.52	155.92	161.29	125.30	159.80
	worst	–	–	–	–	–	11202	10603	10630	10952	11361
Ti2	*best*	10483	12124	**12369**	12006	**12369**	**12369**	**12369**	**12369**	**12369**	**12369**
	mean	10483	11326.3	12081.6	11485.3	11875.9	11994.36	11769.68	11451.76	11939.16	12041.94
	std. dev.	0.00	417.00	193.79	248.33	336.94	436.81	348.00	258.20	421.00	428.71
	worst	–	–	–	–	–	11274	10988	10924	11328	11374
Ti3	*best*	11508	12841	13047	12308	13024	**13696**	12864	12250	13503	**13696**
	mean	11508	12236.6	12522.8	11667.9	12277.5	13204.26	12063.71	11899.68	12837.27	13273.46
	std. dev.	0.00	198.18	201.35	177.14	234.24	366.56	245.51	132.04	366.16	340.21
	worst	–	–	–	–	–	12339	11570	11551	12119	12385
Ti4	*best*	8621	10920	10602	10376	10547	**11298**	10609	10263	10973	**11298**
	mean	8621	10351.5	10150.6	9684.33	10085.4	10801.41	9975.19	9924.61	10514.76	10745.46
	std. dev.	0.00	208.08	152.91	184.84	160.60	205.76	190.47	99.79	266.90	188.77
	worst	–	–	–	–	–	10195	9539	9702	9912	10031
Ti5	*best*	9961	10994	11158	10269	11152	**11568**	11034	10584	11395	**11568**
	mean	9961	10640.1	10775.9	9957.09	10661.3	11317.99	10325.56	10342.51	10857.71	11212.94
	std. dev.	0.00	126.84	116.80	141.48	149.84	182.82	208.30	185.59	226.16	202.85
	worst	–	–	–	–	–	10690	9876	9991	10147	10740
Ti6	*best*	9618	11093	10528	10051	10528	11517	10403	9988	11377	**11763**
	mean	9618	10190.3	9897.92	9424.15	9832.32	10899.20	9648.44	9680.22	10473.56	10984.71
	std. dev.	0.00	249.76	186.53	197.14	232.72	300,36	197.79	173.53	333.75	310.48
	worst	–	–	–	–	–	10281	9182	9289	9775	10438
Ti7	*best*	8672	9799	10085	9235	9883	**10483**	9997	9196	10022	**10483**
	mean	8672	9432.82	9537.5	8904.94	9314.57	10013.43	9127.65	8872	9604.43	9929.36
	std. dev.	0.00	163.84	184.62	111.85	191.59	202.40	222.36	94.27	224.85	191.58
	worst	–	–	–	–	–	9519	8732	8697	8956	9378
Ti8	*best*	8064	9173	9456	8932	9352	**10338**	9099	9172	9738	10302
	mean	8064	8703.66	9090.03	8407.06	8846.99	9524.98	8558.96	8320.16	9005.17	9442.26
	std. dev.	0.00	154.15	156.69	148.52	210.91	286.16	204.56	183.58	248.96	293.21
	worst	–	–	–	–	–	8816	8128	8012	8231	8811
Ti9	*best*	9559	10311	10823	10357	10728	11094	10498	9964	10823	**11097**
	mean	9559	9993.16	10483.4	9615.37	10159.4	10687.62	9895.97	9657.87	10354.50	10663.88
	std. dev.	0.00	117.73	228.34	151.41	198.49	168.06	191.75	124.64	186.68	186.29
	worst	–	–	–	–	–	10201	9570	9454	9878	10222
Ti10	*best*	8157	9329	9333	8799	9218	10104	9057	8846	9649	**10117**
	mean	8157	8849.46	9085.57	8347.82	8919.64	9383.28	8599.17	8426.75	9082.82	9386.52
	std. dev.	0.00	141.84	115.62	122.65	168.90	241.01	185.78	127.23	220.99	233.82
	worst	–	–	–	–	–	8834	8208	8193	8619	8870

$$W(A) = \sum_{e \in \cup_{i \in A} S_i} w_e \leq C, \quad A \subseteq \aleph \tag{2}$$

where w_e is the weight of the eth element in the union set of the selected items, p_i is the profit of ith item in the subset A and C is the knapsack capacity. As formulated by Arulselvan (2014), SUKP is comprised of a set of elements $U = \{1, \ldots, n\}$ and a set of items $\aleph = \{1, \ldots, m\}$, where each item in set $\aleph$ ($i = 1, \ldots, m$) corresponds to a subset of elements, represented by S_i, with a non-negative profit denoted by $p : \aleph \rightarrow \mathbb{R}^+$. Each of the elements has non-negative weight $w : U \rightarrow \mathbb{R}^+$. For an arbitrary subset $A \subseteq \aleph$, the total weight of the union of subset A is defined as $W(A) = \sum_{e \in \cup_{i \in A} S_i} w_e$ and profit of A is denoted by $P(A) = \sum_{i \in A} p_i$. The aim here is to find a subset of items $\aleph^* \subseteq \aleph$ such that the total profit $P(\aleph^*)$ is maximized and $W(\aleph^*)$ does not violate knapsack capacity.

He et al. (2018) present another mathematical formulation for this problem. In this formulation (Eqs. (3)–(4)), $Y = [y_1, y_2, \ldots, y_m] \in \{0,1\}^m$ denotes an m-dimensional 0–1 solution vector that is capable of representing all possible solutions. If $A_Y = \{i | y_i \in Y, y_i = 1, 1 \leq i \leq m\} \subseteq \aleph$, then for an arbitrary $i = 1, 2, \ldots, m$, $y_i = 1$ if and only if $i \in A_Y$.

$$maximize\ f(Y) = \sum_{i=1}^{m} y_i p_i \quad s.t. \tag{3}$$

$$W(A_Y) = \sum_{j \in \cup_{i \in A_Y} U_i} w_j \leq C \tag{4}$$

SUKP is introduced by Goldschmidt et al. (1994). Goldschmidt et al. (1994) employ a dynamic programming approach to solve the SUKP. Later, Lister et al. (2010) address dynamic character caching as an SKUP. Some crucial proofs are reported by Arulselvan (2014). In the same year, Riondato and Vandin (2014) solve the SUKP to obtain an upper bound for their related problem. Several approximation methods are presented by Taylor (2016). Su and Zhou (2016) present a reduced modification for the SUKP. Virtual machine selection problem in cloud computing systems is modelled as an SUKP by Diallo et al. (2017). Jagiello and Taylor (2017) report an interesting application that models a budget scenario problem as SUKP. He et al. (2018) employ an artificial bee colony algorithm, enhanced by a greedy heuristic that improves solutions while handling infeasibility. Ozsoydan (2019), Gölcük and Ozsoydan (2020) and Baykasoğlu et al. (2020) propose some modifications for several metaheuristic algorithms to solve a number of binary optimization problems including the SUKP. Lin et al. (2019) propose an extension of PSO that is hybridized by a trajectory based metaheuristic algorithm. In the same year, another neighbourhood based metaheuristic algorithm is reported by Wei and Hao (2019). The proposed algorithm iterates through two complementary search phases: a local optima exploration phase to discover local optima and a local optima avoiding phase to search for the unexplored regions. Next year, Wu and He (2020) report a novel metaheuristic algorithm, which uses an improved solution encoding mechanism to solve the SUKP. Wei and Hao (2021a,b) propose neighbourhood based metaheuristic approaches employing Tabu Search Algorithm for the SUKP. Finally, Dahmani

Table 5
Final cumulative average Q-learning parameters over all runs.

#	Optimizers								
	GA^*			PSO			gbPSO		
	Invoke	Reward	Rmp.	Invoke	Reward	Imp.	Invoke	Reward	Imp.
Fi1	31.09	4.20	2.62	28.80	2.31	1.89	40.11	10.80	7.52
Fi2	31.25	3.27	2.91	29.86	1.95	1.96	38.89	7.59	7.18
Fi3	58.59	4.17	3.05	55.32	1.65	1.37	86.09	25.71	17.10
Fi4	62.76	5.10	5.28	57.22	2.49	4.34	80.02	19.53	16.31
Fi5	91.33	4.11	2.97	86.07	1.98	1.69	122.60	28.23	16.84
Fi6	93.03	6.33	4.82	87.30	2.76	2.96	119.67	24.84	18.29
Fi7	121.73	5.16	2.80	114.84	2.46	2.04	163.43	38.07	18.77
Fi8	123.07	5.01	2.95	118.82	2.58	2.48	158.11	29.25	18.43
Fi9	154.51	5.04	2.40	151.15	3.24	2.70	194.34	33.09	12.67
Fi10	157.79	5.73	2.88	150.45	2.94	2.29	191.76	28.56	13.20
Si1	30.37	3.81	2.94	30.26	2.85	3.10	39.37	10.35	8.30
Si2	29.44	3.60	3.86	29.31	2.40	2.78	41.25	10.59	12.07
Si3	59.40	3.96	3.09	57.62	2.46	2.54	82.98	23.49	16.32
Si4	59.57	4.32	4.06	57.28	2.61	3.38	83.15	21.39	18.47
Si5	92.68	5.73	3.82	88.01	4.02	4.48	119.31	26.61	15.26
Si6	90.26	4.11	3.33	89.55	2.82	2.40	120.19	26.25	17.99
Si7	121.77	3.90	2.21	118.93	3.00	1.94	159.30	31.92	13.79
Si8	126.18	4.98	3.87	117.32	1.50	1.06	156.50	28.08	17.57
Si9	156.87	5.97	2.60	152.07	3.60	2.52	191.06	30.51	11.37
Si10	157.62	5.55	2.95	149.12	2.16	1.44	193.26	32.49	15.60
Ti1	29.47	4.65	3.29	27.33	1.95	1.96	43.20	16.89	9.61
Ti2	27.92	3.00	3.48	27.73	1.77	2.13	44.35	14.73	15.40
Ti3	59.94	4.41	2.97	54.66	2.13	2.11	85.40	27.66	16.05
Ti4	60.41	5.01	3.70	59.63	3.45	5.83	79.96	20.79	17.33
Ti5	90.85	3.30	1.57	86.81	2.01	1.43	122.34	26.64	12.55
Ti6	91.78	3.84	2.79	89.85	4.05	4.30	118.37	25.11	17.19
Ti7	123.41	4.41	2.29	117.59	2.04	1.12	159.00	31.44	13.75
Ti8	126.48	4.56	2.75	117.92	2.01	1.67	155.60	26.91	15.94
Ti9	157.24	4.98	2.07	147.00	3.03	2.23	195.76	34.98	13.22
Ti10	156.89	5.22	3.12	151.64	2.85	2.11	191.47	28.71	13.71
mean	92.45	4.58	3.11	88.31	2.56	2.47	119.22	24.70	14.59

et al. (2022) analyse the effects of backtracking strategy, which is an enhancement technique adopted by a population based approach. The authors use the SUKP as the case study.

As one can see from the related studies, employing machine learning methods along with promising metaheuristic algorithms is still lacking. In this context, with the additional reported enhancement methods such as TRIM, initial solution generation technique and the proposed particular modifications for binary optimization problems, this study is expected to fill a gap in the related literature.

3. Proposed solution approaches

Since SUKP is a binary optimization problem, all used optimizers should be adapted to binary space. The details of all these procedures are presented in the following.

3.1. Solution encoding

In SUKP, number of items is denoted by m. Therefore, an m dimensional 0–1 vector, where 1s and 0s represent the assigned and discarded items, respectively, is used as the solution vector in the present study. Moreover, a population is comprised of a number of solution vectors represented by population size (*popSize*). Therefore, a population in the present study is a matrix of size $popSize \times m$.

3.2. Particle swarm optimization

Introduced by Eberhart and Kennedy (1995) and Kennedy and Eberhart (1995), PSO uses Eqs. (5)–(6), where current position (x_{ij}), particle best solution ($pbest$) and global best solution (X_{best}) simultaneously affect the movement speed. In these equations, v_{ij}, w, c_1 and c_2 represent speed of the ith particle at the jth dimension, the inertia coefficient and the accelerating coefficients, respectively. A pseudo code for the canonical PSO is presented by Algorithm 1.

$$v_{ij} = wv_{ij} + c_1 \times rand \times \left(pbest_{ij} - x_{ij}\right) + c_2 \times rand \times \left(x_{best,j} - x_{ij}\right) \quad (5)$$

$$x_{ij} = x_{ij} + v_{ij} \quad (6)$$

Algorithm 1. A pseudo code for the canonical PSO.

```
initialize PSO
generate initial particles
while (termination criterion not met) do
    for (i=1 to popSize)
        for (j=1 to m)
            evaluate v_ij via Eq. 5
            update x_ij via Eq. 6
        end
        evaluate new solutions
        update pbest and x_best if necessary
    end
end
print x_best
** popSize :population size
** m: dimension of particles
```

In order to adopt PSO in binary spaces, as formulated by Eqs. (7)–(8), logistics sigmoid function (s-shaped transfer function) is used in the present study. In these equations, x_{ij}, b_{ij} and $rand()$ represent the jth dimension of the ith particle, its corresponding binary value and a uniform random number $\in (0, 1)$, respectively.

$$T\left(x_{ij}\right) = \frac{1}{1 + e^{-x_{ij}}} \quad (7)$$

Table 6
A comparison with the state-of-the-art for the first type of instances.

#	Results	DE	GWO	IntAgents	Djaya	GWO_{fbd}	GWO_{rbd}	HBPSO/TS	QL	gbPSO
Fi1	*best*	13044	13044	**13283**	**13283**	13089	**13283**	**13283**	13251	**13283**
	mean	12810.27	12901.47	13061.02	13076.00	13041.37	13065.93	13277.03	13049.88	13049.42
	std. dev.	137.78	126.35	44.08	66.61	31.17	70.02	–	27.65	27.96
	worst	12517	12634	13044	13044	–	–	–	13044	13044
Fi2	*best*	12237	12237	12274	12274	12274	12274	12348	**12479**	12348
	mean	11968.53	11991.31	12074.84	12192.50	12079.40	12053.57	12262.25	12101.41	12108.39
	std. dev.	161.53	138.89	86.37	70.25	93.34	81.99	–	87.80	80.14
	worst	11571	11571	11855	11985	–	–	–	11912	11985
Fi3	*best*	12936	13074	13502	13405	13405	13405	**13521**	13405	13405
	mean	11767.97	11840.70	13226.28	13306.60	13282.30	13280.78	13521.0	13075.43	13308.94
	std. dev.	277.74	267.05	150.92	60.96	102.88	123.63	–	209.89	95.09
	worst	11260	11383	12733	13137	–	–	–	12438	12818
Fi4	*best*	13403	13246	14044	**14215**	**14215**	**14215**	**14215**	14044	**14215**
	mean	12455.90	12528.33	13441.06	13660.20	13464.35	13479.99	13952.8	13309.32	13595.80
	std. dev.	287.04	245.54	324.96	274.76	358.97	358.56	–	294.52	365.53
	worst	11785	11968	12802	13202	–	–	–	12727	12934
Fi5	*best*	9899	9991	11335	10934	11413	11335	**11563**	10952	11335
	mean	9334	9453.29	10576.10	10703.20	10707.54	10684.17	11401.93	10281.50	10719.21
	std. dev.	172.59	140.96	281.13	112.95	230.46	242.43	–	255.44	192.55
	worst	9062	9210	9884	10390	–	–	–	9774	10116
Fi6	*best*	10593	11216	12247	12245	12402	12259	**12607**	12245	12245
	mean	9855.63	9853	11490.26	12037.50	11646.23	11606.32	12457.73	11096.13	11660.46
	std. dev.	223.10	239.01	518.81	296.02	517.63	492.99	–	576.70	481.10
	worst	9495	9494	10072	11057	–	–	–	10049	10438
Fi7	*best*	9637	9548	**11484**	11337	**11484**	**11484**	**11484**	11242	**11484**
	mean	9076.72	9124.25	10734.62	11062.00	10884.49	10880.22	11439.75	10431.49	10845.05
	std. dev.	141.14	136.02	371.37	273.63	396.92	386.79	–	395.26	376.94
	worst	8850	8890	9793	10167	–	–	–	9492	9895
Fi8	*best*	8935	9007	10710	10431	10710	10757	**11209**	10419	10526
	mean	8478.20	8621.83	9735.00	10017.90	9894.54	9900.01	11031.7	9580.06	9846.91
	std. dev.	151.21	146.45	370.44	207.98	329.34	325.03	–	338.07	354.96
	worst	8209	8282	8863	9445	–	–	–	8843	8806
Fi9	*best*	10182	10341	11722	11722	11722	**11771**	11716	11414	**11771**
	mean	9868.51	10042.74	11111.63	11269.40	11276.49	11338.26	11484.65	10837.59	11217.69
	std. dev.	99.02	128.52	355.18	275.37	347.99	351.46	–	253.40	354.47
	worst	9642	9780	10457	10689	–	–	–	10332	10577
Fi10	*best*	8760	8890	10059	9770	**10194**	**10194**	**10194**	9729	10022
	mean	8294.30	8347.02	9165.26	9354.28	9339.80	9398.07	9896.875	8993.48	9222.81
	std. dev.	125.05	119.39	282.55	212.69	252.98	266.46	–	216.57	241.45S
	worst	8105	8177	8647	9042	–	–	–	8488	8786

$$b_{ij} = \begin{cases} 0 \; if \; rand() < T\left(x_{ij}\right) \\ 1 \; if \; rand() \geq T\left(x_{ij}\right) \end{cases} \tag{8}$$

3.3. Genetic algorithm

The standard GA (Holland, 1975) starts with randomly generated chromosomes that form up a population. Next, according to a user-supplied crossover rate (*xRate*) parameter, some selected chromosomes undergo crossover operations either randomly or via roulette wheel procedure. Thus, offspring population is obtained. This population is subsequently exposed to a number of mutations defined by another user-supplied parameter, namely, mutation rate (*mutRate*). After evaluating the fitness values of all mutant offspring chromosomes, finally, natural selection (roulette wheel, tournament, etc.) is performed to generate the next generation. A sample pseudo code for GA is illustrated by Algorithm 2.

The GA adopted in present study employs a uniform-based crossover operator, of which the details are presented by Algorithm 3. According to this operator, chromosomes in population undergo a bit-wise crossover operation. For each bit of the chromosomes, a random number $rand() \in (0, 1)$ is generated. If this number is less than or equal to $\theta \in [0, 1]$, which is a user-supplied parameter, the offspring inherits the corresponding allele from the current chromosome, otherwise it is inherited from the randomly selected chromosome.

Algorithm 2. A pseudo code for the canonical GA.

```
initialize GA
generate initial chromosomes
while (termination criterion not met) do
        select the chromosomes for reproduction
        invoke crossover
        invoke mutation
        evaluate fitness of the offspring
        invoke natural selection
        invoke elitism **optional
end
print the best solution
```

Subsequent to uniform-based crossover, bit-flip mutations are conducted. It is worth stressing that mutation operator is performed only to offspring population. According to this operator, a number of randomly selected bits in the offspring population is flipped. Next, a roulette

Table 7
A comparison with the state-of-the-art for the second type of instances.

#	Results	DE	GWO	IntAgents	Djaya	GWO_{fbd}	GWO_{rbd}	HBPSO/TS	QL	gbPSO
Si1	*best*	13860	13814	**14044**	**14044**	**14044**	**14044**	13990	**14044**	**14044**
	mean	13566.22	13639.05	13767.23	13912.50	13861.35	13847.86	13952.8	13851.28	13865.98
	std. dev.	136.88	71.12	131.59	84.55	84.62	100.33	–	78.86	84.19
	worst	13249	13394	13422	13767	–	–	–	13664	13664
Si2	*best*	13407	13407	**13508**	**13508**	**13508**	**13508**	13498	**13508**	**13508**
	mean	13040.65	13112.76	13003.62	13439.10	13312.57	13297.37	13293.45	13343.37	13375.41
	std. dev.	306.71	268.92	375.74	44.86	189.12	172.16	–	176.81	154.28
	worst	12321	12358	11739	13407	–	–	–	12809	12860
Si3	*best*	11210	11046	**12522**	**12522**	12350	**12522**	**12522**	12196	**12522**
	mean	10431.36	10445.50	11586.26	12171.60	11852.44	11906.97	12498.98	11408.12	11995.98
	std. dev.	225.38	182.65	419.09	220.68	371.57	382.91	–	387.73	320.78
	worst	10023	10072	10597	11,608	–	–	–	10626	11150
Si4	*best*	11067	11365	11911	12187	11993	**12317**	**12317**	11911	**12317**
	mean	10325.11	10483.14	11288.25	11746.00	11612.07	11594.90	12299.6	11458.29	11772.21
	std. dev.	251.80	281.47	410.54	181.18	217.13	301.10	–	313.89	200.78
	worst	9877	10003	10258	11123	–	–	–	10404	11184
Si5	*best*	11757	11632	12695	12695	**12784**	12695	12736	12644	12695
	mean	10856.26	11031.23	12310.19	12569.30	12441.00	12446.21	12715.58	12019.86	12404.31
	std. dev.	226.15	193.98	238.32	114.13	247.67	227.47	–	222.38	250.50
	worst	10455	10666	11812	12282	–	–	–	11553	11865
Si6	*best*	9745	9566	11425	11113	11425	11425	**11585**	11282	11425
	mean	9151.45	9229.37	10384.00	10701.90	10632.71	10648.53	11532.08	10242.21	10555.03
	std. dev.	193.86	128.22	378.42	153.66	345.63	328.13	–	347.52	326.82
	worst	8861	8895	9528	10223	–	–	–	9528	9700
Si7	*best*	10116	10158	**11531**	11310	**11531**	**11531**	11433	11158	11432
	mean	9554.85	9616.85	10756.92	10914.80	10961.25	10964.98	11399.65	10465.86	11012.87
	std. dev.	125.05	124.30	250.56	216.47	258.47	276.16	–	246.03	236.15
	worst	9329	9421	10210	10422	–	–	–	9844	10562
Si8	*best*	8845	8685	10927	10915	10927	10927	**11325**	10520	10927
	mean	8337.25	8304.50	9608.07	9969.90	9849.04	9873.27	11321.0	9283.62	9742.38
	std. dev.	155.71	117.90	363.72	287.61	343.90	373.70	–	358.36	403.36
	worst	8025	8063	8936	9228	–	–	–	8619	8922
Si9	*best*	10071	10073	10960	10960	10921	10960	10973	10888	**10991**
	mean	9662.80	9775.79	10610.53	10703.50	10716.55	10742.98	10832.53	10435.59	10662.23
	std. dev.	107.20	95.36	169.73	105.18	140.87	130.05	–	178.33	168.62
	worst	9495	9620	10053	10457	–	–	–	9918	10189
Si10	*best*	9164	8999	10381	10176	**10194**	**10194**	10086	9938	**10194**
	mean	8568.84	8583.18	9578.89	9801.50	9758.61	9737.48	9797.25	9393.54	9637.06
	std. dev.	128.29	129.58	278.06	222.21	243.59	272.51	–	254.37	255.63
	worst	8294	8398	8669	9404	–	–	–	8755	9071

wheel that is based on the enlarged sampling pool is performed to select the chromosomes of the next generation. Finally, elitism is applied.

Algorithm 3. Adopted uniform-based crossover.

```
X_i  **current solution vector
R    **randomly selected solution vector
O    **offspring
for (j=1 to m)
    if rand() ≤ θ
        o_j ← x_ij
    else
        o_j ← r_j
    end
end
```

3.4. gbPSO

GA, adopted in the present study can work on binary spaces via employed uniform-based crossover and bit-flip mutation operators without a transfer function. However, canonical PSO require transfer functions to work on binary spaces. It is clear that transfer function (Eqs. (7)–(8)) is a probabilistic function and thus, it is not guaranteed that two identical continuous vectors result in the same binary vector. This yields to an undesirable circumstance and spatial distance. In order to avoid this, the method of Ozsoydan and Baykasoglu (2019) is adopted and is further extended in the present study.

The proposed method makes use of the uniform-based crossover to generate a new particle, which inherits genes from its current solution vector, personal best solution vector and global best solution vector according to user-supplied probabilities. The proposed method further extends this so as to be able to utilize information of other solution vectors in population and to increase diversity. In this regard, three parameters denoted by $b_1, b_2, b_3 \in [0,1]$ and $b_1 \leq b_2 \leq b_3$ are defined first. Next, while generating a new particle, a uniform random number $rand \in (0,1)$ is generated for each dimension individually. If the generated random number for the corresponding gene is less than b_1, the new particle inherits that gene from a randomly selected particle in population. If $rand$ is greater than or equal to b_1 and is less than b_2, that gene is inherited from the current solution vector. If $rand$ is greater than or equal to b_2 and is less than b_3, that gene is inherited from the particle-best solution vector. Otherwise, ($rand \geq b_3$) the corresponding gene of the new particle is inherited from the global best solution vector. This same procedure is carried out for all dimensions. Thus, a swarm intelligence algorithm that adopts evolutionary operators and works in the notion of PSO is obtained. Therefore, it is referred to as genetic-based PSO (gbPSO) in the current work. A pseudo code for the gbPSO is illustrated by Algorithm 4.

There are similar modifications in the related literature. In one of them, Anghinolfi and Paolucci (2009) propose a PSO extension for job scheduling problems. There are reported analogous methods for

similar scheduling-related problems addressed in the literature (Xia and Wu, 2005; Lian et al., 2006; Pan et al., 2008; Tseng and Liao, 2008). Marinakis and Marinaki (2010), Marinakis et al. (2010) propose a hybridization of GA and PSO for routing problems. In their approach, particles utilize a shared memory with a predetermined probability. Kuo et al. (2012) report another analogous approach that crosses a particle sequentially with the *particle best* and *global best*. Another GA-PSO hybrid for routing problems is proposed by Ozsoydan and Sipahioglu (2013). Finally, Ozsoydan (2019) proposes another modified swarm intelligence based optimizer that makes use of hyper-heuristics.

Algorithm 4. A pseudo code for gbPSO.

```
initialize gbPSO
generate initial particles
while (termination criterion not met) do
    for (i=1 to popSize)
        for (j=1 to m)
            generate rand ∈ (0,1)
            if rand < b1
                o_ij ← r_j
            end if
            if rand ≥ b1 and rand < b2
                o_ij ← x_ij
            end if
            if rand ≥ b2 and rand < b3
                o_ij ← pbest_ij
            end if
            if rand ≥ b3
                o_ij ← x_best,j
            end if
        end for
        O_i ← mutation(O_i)
        O_i ← S-GROA(O_i)
        update X_best and pbest_i if necessary
    end for
    TRIM()
end while
print the best-found solution
```

3.5. Auxiliary procedures

3.5.1. Mutation(), TRIM and avoiding loss of diversity

Preliminary work reveals that a binary population might be exposed to premature convergence due to loss diversity that also yields to local optima issue. In this regard, the method of Baykasoglu (2012) and Ozsoydan and Baykasoglu (2019) is also adopted by all developed algorithms in the present study. This method is represented by function *mutation* It is worth stressing that the function *mutation*() mentioned here is not that of the GA's. It is an additional procedure that is invoked by all algorithms (including GA) whenever a new solution vector is generated.

In the function *mutation*(), a number of bits are flipped. The number of bits to flip ($rBits$) is exponentially decreased throughout generations. Therefore, $r^t \in [0,1]$, which denotes the ratio of the bit-flip operations, is defined first. This parameter is decreased throughout generations via Eq. (9), where φ is another parameter that controls the speed of decrement. Next, it is multiplied by dimension (m) and the obtained value is rounded up to obtain the total number of bits to flip (Eq. (10)). Finally, randomly selected $rBits^{t+1}$ locations in the newly generated particles are flipped.

$$r^{t+1} = r^t - e^{-t/(t+1)} \times \varphi \times r^t \tag{9}$$

$$rBits^{t+1} = \left\lceil r^{t+1} \times m \right\rceil \tag{10}$$

The function *mutation*() has the capability of preserving diversity among the newly generated individuals. Thus, it helps avoiding local optima to some extent. However, loss of population diversity remains as an issue to be handled. Therefore, algorithms adopted in the present study employ an additional mechanism, namely, triggered random immigrants (TRIM) (Grefenstette, 1992). In this method (Wang et al., 2009), whenever the average diversity of a population that is evaluated based on the hamming distances from the best-found-solution goes below a user-supplied threshold, some random individuals replace the same number of existing solution vectors. The mentioned method is formulated by Eqs. (11)–(12), where $X_i = (x_{i1}, x_{i2}, \ldots, x_{im})$ and $X_k = (x_{k1}, x_{k2}, \ldots, x_{km})$ represent two different individuals from population. While Eq. (11) formulates the hamming distance between two individuals, mean distance from the best-found solution is formulated by Eq. (12). Finally, if ξ goes below the parameter Y, this mechanism is triggered and a group of randomly generated ψ individuals replaces the worst ψ individuals in population.

$$d\left(X_i, X_k\right) = \frac{\sum_{j=1}^{m} \left| x_{ij} - x_{kj} \right|}{m} \quad i \neq k \tag{11}$$

$$\xi = \frac{\sum_{i=1}^{popSize} d(X_i, X_{best})}{popSize} \tag{12}$$

3.5.2. Generating initial population

In order to initialize the first population with better individuals, in this paper, average weights of all individual items are evaluated first. Next, obtained average weight values are proportioned to the profit values of the corresponding item and thus, the well-known profit/weight ratios for all items are obtained. It is clear that greater values of this ratio point out more attractive items. Therefore, items are attempted to be assigned to knapsack in decreasing order of these ratios considering the capacity constraint. Finally, obtained 0–1 m dimensional vector is assigned as the first individual of the initial population. The second individual of the initial population is obtained by applying the *mutation*() function (Eqs. (9)–(10)) to this individual. Thus, the second individual is indeed a perturbed vector of the first individual. In order to avoid unnecessarily too much improvement on the first population that might also yield to local optima and premature convergence, the rest of the population (*popSize*-2) is comprised of randomly generated 0–1 solution vectors.

3.5.3. Handling infeasibility

Another auxiliary procedure that is adopted by all optimizers used in the present study is S-GROA (He et al., 2018), which is used to repair both incomplete and infeasible solutions. This method makes use of the profit and resource consumption values of items and sorts them first. Subsequently, it attempts to eliminate items that are violating the knapsack capacity in case of infeasibility. It is clear that after such an elimination, an amount of unused capacity might be left. This method then attempts to use the remaining capacity by filling the knapsack by more profitable items. Thus, even if a solution is feasible, S-GROA utilizes the remaining capacity by assigning the possible items among from the sorted items list.

3.6. Q-learning to reinforce optimizers

Q-learning (Watkins and Dayan, 1992), which is a model-free reinforcement learning algorithm, has five components, namely, agents, environment, actions, state and reward. Agents represent the entity that chooses actions at some states. Environment can be considered as a space, which agents interact with. Actions represent various alternatives or policies that can be chosen to fulfil a task. States can be considered as positions, where agents are located to choose some actions so as to jump to the next stage. Finally, reward and penalty are scalars that are assigned to agent according to the achievement of the chosen action. At each state, an agent performs actions and receives

Table 8
A comparison with the state-of-the-art for the third type of instances for SUKP.

Instance	Results	DE	GWO	IntAgents	Djaya	GWO_{fbd}	GWO_{rbd}	HBPSO/TS	QL	gbPSO
Ti1	*best*	11351	11478	**12045**	**12045**	**12045**	**12045**	**12045**	11619	**12045**
	mean	10899.48	10877.71	11419.75	11570.60	11441.23	11430.44	12045.0	11371.96	11525.48
	std. dev.	166.21	178.26	140.77	177.86	111.72	127.56	–	125.30	159.80
	worst	10576	10526	11052	11373	–	–	–	10952	11361
Ti2	*best*	**12369**	**12369**	**12369**	**12369**	**12369**	**12369**	**12369**	**12369**	**12369**
	mean	11346.21	11320.16	11885.21	12318.00	11917.83	11942.93	12335.75	11939.16	12041.94
	std. dev.	267.25	289.90	431.67	181.92	442.25	418.72	–	421.00	428.71
	worst	10924	10834	11374	11374	–	–	–	11328	11374
Ti3	*best*	12548	12155	**13696**	**13696**	13647	**13696**	**13696**	13503	**13696**
	mean	11633.11	11748.11	13084.52	13350.20	13121.23	13125.85	13690.8	12837.27	13273.46
	std. dev.	201.98	147.36	388.39	182.56	365.41	367.06	–	366.16	340.21
	worst	11290	11408	12210	12749	–	–	–	12119	12385
Ti4	*best*	10201	10201	**11298**	**11298**	**11298**	**11298**	**11298**	10973	**11298**
	mean	9640.42	9774.91	10780.14	10828.90	10871.49	10819.34	11298.0	10514.76	10745.46
	std. dev.	173.39	136.23	239.61	191.76	240.32	239.52	–	266.90	188.77
	worst	9266	9510	10195	10306	–	–	–	9912	10031
Ti5	*best*	10468	10505	11568	11568	11568	11568	**11802**	11395	11568
	mean	9882.77	10031.75	11205.72	11327.70	10001.33	10014.47	11787.2	10857.71	11212.94
	std. dev.	180.87	145.26	258.49	166.91	174.62	215.35	–	226.16	202.85
	worst	9588	9741	10001	10791	–	–	–	10147	10740
Ti6	*best*	9952	9953	11517	11401	11590	**11763**	11538	11377	**11763**
	mean	9411.61	9512.99	10747.33	11025.90	10470.33	10871.49	11536.78	10473.56	10984.71
	std. dev.	173.38	192.61	334.25	208.08	340.47	332.09	–	333.75	310.48
	worst	9063	9183	9898	10582	–	–	–	9775	10438
Ti7	*best*	9175	9240	10326	10414	10397	**10483**	10465	10022	**10483**
	mean	8817.33	8812.93	9892.17	10017.00	10043.23	9902.72	10340.28	9604.43	9929.36
	std. dev.	113.89	106.32	179.19	141.15	163.95	180.32	–	224.85	191.58
	worst	8577	8596	9327	9676	–	–	–	8956	9378
Ti8	*best*	8562	8917	10131	10302	10302	10302	**10506**	9738	10302
	mean	8192.20	8236.63	9339.67	9565.72	9472.39	9455.24	10339.45	9005.17	9442.26
	std. dev.	146.74	154.02	288.88	237.90	242.25	261.75	–	248.96	293.21
	worst	7908	8026	8783	9200	–	–	–	8231	8811
Ti9	*best*	9772	9930	11094	10971	10989	11097	**11115**	10823	11097
	mean	9520.33	9587.57	10603.53	10754.80	10702.72	10725.26	10872.38	10354.50	10663.88
	std. dev.	86.09	125.92	204.99	112.69	154.92	160.001	–	186.68	186.29
	worst	9310	9335	10079	10479	–	–	–	9878	10222
Ti10	*best*	8634	8674	10104	9715	10104	10104	10104	9649	**10117**
	mean	8281.76	8346.20	9259.36	9467.80	9462	9455.24	10008.13	9082.82	9386.52
	std. dev.	113.39	128.69	268.33	106.55	229.88	261.74	–	220.99	233.82
	worst	8084	8156	8620	9122	–	–	–	8619	8870

a reward or penalty. In the present study, the proposed Q-learning algorithm aims to select one of the optimizers at each iteration of search. The architecture of the used Q-learning algorithm is illustrated by Fig. 1.

In this proposed approach, number of algorithms (alternatives) is denoted by *nAlgo* (Gölcük and Ozsoydan, 2021). Accordingly, the set of states and actions are represented by $S = \{s_1, s_2, \ldots, s_{nAlgo}\}$ and $A = \{a_1, a_2, \ldots, a_{nAlgo}\}$, respectively. The immediate reward acquired by any of the algorithms (actions) is denoted by $reward_{t+1}$.

In order to estimate the best state–action pairs at each iteration, the Q-table, where cumulative rewards are recorded, is used. The total cumulative reward for the (s_t, a_t) pair is computed by Eq. (13), where α_t and γ are learning rate and discount factor, respectively. In Eq. (13), the parameter α controls the balance of exploration and exploitation.

$$Q_{t+1}(s_t, a_t) = Q_t(s_t, a_t) + \alpha_t \left[reward_{t+1} + \gamma \max_a Q_t(s_{t+1}, a) - Q_t(s_t, a_t) \right] \tag{13}$$

In a wide range of studies, the parameter γ is fixed to 0.8 while α is gradually decreased from 1 as formulated by Eq. (14), where t and *maxIter* represent the current and the maximum number of iterations, respectively. Moreover, the immediate reward function for a maximization problem is formulated by Eq. (15), where either positive or negative reinforcement is computed.

$$\alpha_t = 1 - \left(0.9 \times \frac{t}{maxIter}\right) \tag{14}$$

$$reward_{t+1} = \begin{cases} +1 \; if \; f_{new} - f_{X_{best}} > 0 \\ 0 \; if \; f_{new} - f_{X_{best}} = 0 \\ -1 \; if \; f_{new} - f_{X_{best}} < 0 \end{cases} \tag{15}$$

In order to avoid promoting the greedy behaviour of this algorithm, this study adopts the ε-greedy method (Gölcük and Ozsoydan, 2021) encompassing three different mechanisms (Gosavi, 2015; Mosadegh et al., 2020), while choosing the next action.

In this strategy, the first mechanism has ε % probability of selecting actions at random. Thus, the selected action can easily be computed as formulated by Eq. (16). The second mechanism is invoked with the probability of χ % and chooses the action based on the well-known roulette-wheel selection. Here, the policy is stochastically implemented according to Boltzmann distribution (Jang et al., 1997) as formulated by Eq. (17), where $prob(a_k)$ represents the probability of kth action that undergoes roulette-wheel selection. Finally, the third mechanism is the maximum Q-factor method in which the action index is obtained by Eq. (18). Bringing all algorithms together, the pseudocode of the developed Q-learning algorithm is given by Algorithm 5.

$$\lceil nAlgo \times rand() \rceil \tag{16}$$

$$prob(a_k) = \frac{\exp(Q(s, a_k))}{\sum_k \exp(Q(s, a_k))} \tag{17}$$

$$\arg\max_k (Q(s, a_k)) \tag{18}$$

As one can see from the pseudo code given by Algorithm 5 and the illustration provided by Fig. 1, the proposed algorithm works as

an algorithm recommendation system based on a feedback mechanism. At each iteration of the search, Q-learning mechanism invokes an optimizer according to its previous achievements. Next, it is rewarded or punished. Therefore, a more promising algorithm is more likely to be invoked by this mechanism whereas naïve algorithms are invoked less frequently. Thus, throughout iterations, this mechanism is expected to learn and recommend more successful algorithms.

Algorithm 5. A pseudo code for the proposed Q-learning algorithm.

initialize PSO, GA and gbPSO
generate initial particles
initialize Q-table, $S = \{s_1, s_2, \ldots, s_{nAlgo}\}$, $A = \{a_1, a_2, \ldots, a_{nAlgo}\}$
set $Q(s,a) \leftarrow 0 \ \forall s, a$
while (*termination criterion not met*) **do**
  if $rand \leq \varepsilon$
    $a \leftarrow action_k : k = \lfloor nAlgo \times rand() \rfloor$ ***Eq.16*
  elseif $rand \leq \varepsilon + \chi$
    $a \leftarrow action_{k'} : k' = roulette\left(\frac{\exp(Q(s,a_k))}{\sum_k \exp(Q(s,a_k))}\right)$ ***Eq.17*
  else
    $a \leftarrow action_{k''} : k'' = \arg\max_k\left(Q(s,a_k)\right)$ ***Eq.18*
  end if
  if $a == 1$ ***invoke PSO*
    population=PSO(population)
  elseif $a == 2$ ***invoke GA*
    population=GA(population)
  else ***invoke gbPSO*
    population=gbPSO(population)
  end if
  obtain $reward_t$ ***Eq. 15*
  obtain the maximum Q factor for the next state of $state_{t+1}$
  update $Q(s,a)$ ***Eq. 13*
  $state_{t+1} \leftarrow state_{t+1}$
  evaluate new α_t ***Eq. 14*
end while
print the best-found solution

4. Experimental study

He et al. (2018) introduce three types of SUKP instances, which are also adopted in the present study. These instances are classified according to the number items and the number of elements. In the first category, number of items is greater than number of elements, while they are equal in the second category. Finally, in the third category, number of items is less than number of elements. For fair comparisons with the literature, $popSize$ (number of individuals in a population) is fixed to 20 also in the present study and the termination criterion is used as achieving the maximum number of iterations ($maxIter$), which is equal to $max\{number\ of\ items, number\ of\ elemetns\}$ as proposed by He et al. (2018). All obtained results are evaluated over 100 independent replications. Algorithms are coded on MATLAB. All tests are carried out on a PC with i7-4790K CPU and 32 GB RAM.

4.1. Calibration of algorithm parameters

PSO makes use of the parameters denoted by w, c_1 and c_2. As proposed by Mirjalili et al. (2012), c_1 and c_2 are both fixed to 2 while w is monotonically decreased from 0.9 to 0.4. Initial velocity values are randomly generated between (0,1). As the second optimizer, GA employs three parameters represented by *xRate*, *mutRate* and θ. The effects of these parameters have been tested at various intervals over wide ranges. Finally, it is observed that better results can be achieved when *xRate*, *mutRate* and θ are set to 1.0, 0.005 and 0.4, respectively. The third optimizer gbPSO uses the parameters b_1, b_2 and b_3. The values of the last two parameters are adopted from Ozsoydan and Baykasoglu (2019). Since these parameters are in the interval [0,1], the new parameter b_1 can be obtained by shifting the b_1 and b_2 values of Ozsoydan and Baykasoglu (2019), which correspond to b_2 and b_3, respectively in the present study. However, in order not to exceed 1.0, similar values are used here. In this regard, preliminary work points out that better results can be obtained when b_1, b_2 and b_3 are set to 0.10, 0.15 and 0.65, respectively.

Table 9
Friedman test ranks.

Algorithms	*Average ranks*
PSO	3.75000
*GA**	3.10000
QL	1.95000
gbPSO	1.20000
p-value	0.00000

The parameters of the *mutation*(), denoted by φ and r^1 are fixed to 0.005 and 0.05, respectively (Ozsoydan and Baykasoglu, 2019). As another auxiliary procedure, TRIM uses two parameters represented by Υ and ψ. Preliminary work points out that better results can be achieved when the threshold Υ and the parameter ψ are set to 0.30 and 4, respectively. Thus, when diversity goes below 0.30, the worst 20% of population replaces randomly generated new solution strings. Finally, the parameters of the proposed Q-learning algorithm, denoted by a, γ, ε and χ are calibrated. The parameter a is decreased from 1.0 by using the formulation given by Eq. (14). The values of the remaining parameters are adopted from the related literature. Thus, as suggested by a number of published papers, the parameters γ, ε and χ are set to 0.80, 0.10 and 0.80, respectively.

4.2. Experimental results

The related information about the used instances are presented by Table 1. The first group of instances is denoted by Fi, while the second and the third group of instances are represented by Si and Ti, respectively. According to Table 1, the smallest-scaled instances in each of these categories are comprised of 100 items and 85 elements, 100 items and 100 elements, 85 items and 100 elements, while the greatest-scaled instances are comprised of 500 items and 485 elements, 500 items and 500 elements, 485 items and 500 elements in sequence. The remaining details are reported by He et al. (2018).

All obtained results are presented by Tables 2–4, where *best*, *mean*, *std. dev.* and *worst* represent the best-found solutions, mean of the best-found solutions, standard deviation of the best-found solutions and the worst solution over all replications. The columns *PSO*, *GA**, *QL* and *gbPSO* denote the used optimizers in the present study. The results regarding the rest of the columns *A-SUKP*, *GA*, *BABC*, ABC_{bin} and *binDE* are adopted from He et al. (2018). Finally, the values in bold in these tables represent the best solution.

As one can see from Table 2, the developed gbPSO finds the *best* solutions in 9 out of 10 instances. Regardless of the problem scale, the proposed algorithm seems to be achieving promising results in terms of finding the *best* solutions. Additionally, gbPSO exhibits a good performance also in terms of the *worst* and *std.* values. Another optimizer PSO can be considered as one of the naïve algorithms tested in the present study since it is not found to be capable of achieving promising results in comparison to other tested algorithms. It is also clear that although GA* that can directly work on binary spaces is not found to be as good as gbPSO and QL, it is found to be better than PSO. Finally, QL, which adopts GA*, PSO and gbPSO can also be considered as another promising algorithm.

The results reported in Table 3 reveal that the developed gbPSO is found to be promising also in the second type of instances, where the number of items is equal to the number of the elements. gbPSO finds the *best* solutions in 8 out of 10 instances in this table. In this regard, it can be concluded that regardless of the problem scale, gbPSO seems to be achieving promising results in terms of finding the *best* solutions also in the second type of instances. Moreover, gbPSO exhibits similar performance in terms of the *worst* and *std.* values again. It is clear that PSO can be considered as one of the naïve algorithms particularly

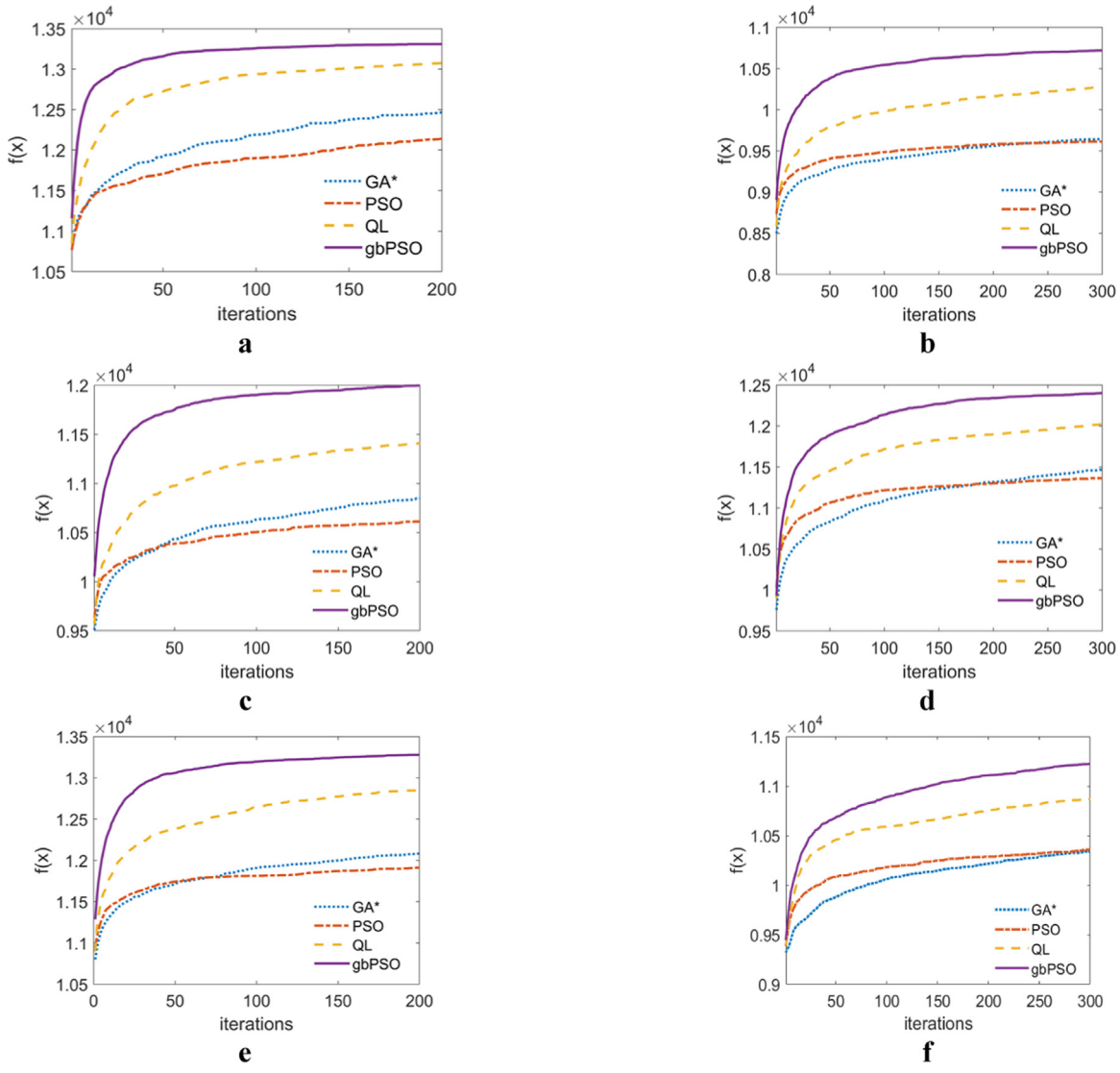


Fig. 2. Mean convergence plots of used optimizers for the instances **(a)** Fi3 **(b)** Fi5 **(c)** Si3 **(d)** Si5 **(e)** Ti3 **(f)** Ti5.

in medium-scaled and large-scaled instances of Table 3. As another optimizer, GA* achieves better results in comparison to PSO. This is in parallel with the previous findings. Finally, QL, which achieves a similar performance to that of the gbPSO's in some of the instances, can also be considered as another promising algorithm. It is positioned between the best and naïve optimizers again in the second type of instances.

Table 4 illustrates the obtained results for the third type of instances of the SUKP. gbPSO is found to be promising again since it finds the *best* solutions in 9 out of 10 instances. Therefore, this algorithm is able to achieve promising results in terms of finding the *best* solutions regardless of the problem scale and instance type. Furthermore, gbPSO exhibits a good performance in terms of the *worst* and *std.* values also in the third type of instances. PSO can be considered as a naïve algorithm since it is not able to achieve promising results in comparison to other tested algorithms again. It is also clear that GA* is not as good as gbPSO and QL, however, it is found to be better than PSO. Another tested algorithm QL, which achieves a similar performance to that of the gbPSO's particularly in some of the instances, can also be considered as another promising algorithm for the third type of instances. Finally, it can be concluded that findings obtained here are in parallel with the previous ones.

As reported before, the developed QL algorithm occasionally invokes some naïve algorithms such PSO and GA*. Therefore, the performance of this algorithm is expected to be positioned between the best and the naïve optimizers tested in the present study. This is also validated by the results of Tables 2–4. Nevertheless, the merit of QL adopted in the present study is indeed the validation and recommendation of the promising algorithms based on the feedback and knowledge gathered throughout all iterations. Knowledge, obtained by QL algorithm is summarized in Table 5, where *invoke*, *reward* and *imp.* columns represent the mean over cumulative values of the number of the times that the related algorithm is invoked by the QL algorithm, the cumulative reward that the related algorithm is assigned with and the relative improvement percentage of the related algorithm, respectively. Therefore, the sum of the *invoke* values of all algorithms for any instance equals to *maxIter* for the related instance. Finally, the last row represents the mean of the *invoke*, *reward* and *imp.* values over all instances.

As one can see from this table, QL tends to invoke gbPSO more frequently in comparison to GA* and PSO. Moreover, GA* is invoked more frequently than PSO. This is an expected result, since, as the results of Tables 2–4 point out, the most promising algorithm is found to be gbPSO, which is followed by GA* that is also followed by PSO. Similarly, according to the mean cumulative *reward* values reported in

Table 10
Results of pairwise comparisons.

Comparisons	*Unadjusted p-values*	*Decision on* H_0	*Adjusted p-values*	*Decision on* H_0
gbPSO vs. PSO	0.00000	*reject*	0.00000	*reject*
*gbPSO vs. GA**	0.00000	*reject*	0.00000	*reject*
QL vs. PSO	0.00000	*reject*	0.00000	*reject*
*QL vs. GA**	0.00056	*reject*	0.003364	*reject*
gbPSO vs. QL	0.02444	*reject*	0.146694	*cannot be rejected*
GA vs. PSO*	0.05117	*reject*	0.307057	*cannot be rejected*

the last row of Table 5, PSO gains the least *reward* over all instances, while gbPSO gains the greatest *reward*. The mean *reward* value of GA* is found to be slightly better than that of the PSO's. In parallel with these findings, gbPSO achieves the best *imp*. value, while GA* has a slightly better *imp*. value than that of the PSO's. Finally, regarding the values illustrated by Table 5, it can be concluded that the QL algorithm also validates and recommends that the developed gbPSO tends to be exhibiting better performance in comparison to the other tested algorithms.

Convergence plots of all used optimizers are depicted by Fig. 2, where each data point represents the mean of the best-found solution until the related iteration over all replications. Due to space limitation, only two arbitrary instances from each benchmark type are selected. In this regard, the convergence behaviours of the algorithms on Fi3, Fi5, Si3, Si5, Ti3 and Ti5, which can be considered as medium-scaled instances, are illustrated.

As one can see from these figures, regardless of the benchmark type, the convergence of gbPSO seems to be more promising than the rest of the algorithms. As expected, QL is positioned between the gbPSO and other tested algorithms. This is also a parallel finding regarding the results of Tables 2–4. Although GA* and PSO seem to be exhibiting a competitive convergence behaviour in some of the instances, it can be put forward that GA* is slightly better than PSO.

The proposed algorithm gbPSO along with QL are further compared to the well-regarded and recently published state-of-the-art algorithms DE, Grey Wolf Optimizer (GWO) (Mirjalili et al., 2014), HBPSO/TS (Lin et al., 2019), intAgents (Ozsoydan, 2019), Djaya (Wu and He, 2020) GWO_{fbd}, GWO_{rbd}, (Gölcük and Ozsoydan, 2020). The parameters of DE and GWO are adopted from Mirjalili et al. (2014). All obtained results for all used benchmarks are presented by Tables 6–8.

According to the results of the benchmarks presented by Tables 6–8, one can put forward that HBPSO/TS, QL and gbPSO exhibit competitive performances and are capable of finding the most remarkable results in terms of the best-found solutions. Similarly, intAgents, Djaya GWO_{fbd} and GWO_{rbd} can also be considered as competitive algorithms, since; they also achieve promising results in some instances of the first, the second and the third benchmarking problems. However, it is clear that as the problem size increases, the promising performances of the algorithms HBPSO/TS, QL and gbPSO become more apparent. Additionally, it is clear that Djaya is found to be performing better in terms of finding the most promising worst solution and the standard deviance. Finally, the standard versions of DE and GWO are found to be naïve algorithms in solving SUKP.

In order to demonstrate what intuition suggests thus far, statistical tests are conducted. It is reported that while comparing the performances of stochastic search algorithms, particularly non-parametric tests are encouraged (Derrac et al., 2011). Since, the safe use of parametric tests is dependent to several conditions, which are not fulfilled in the present study. In this regard, the details of the used non-parametric tests are reported in the following sub-section.

4.3. Statistical analysis

In order to analyse possible significant differences between the tested algorithms, Friedman Test is applied first. Results of the Friedman Test are reported by Table 9. According to the Friedman ranks in Table 9, where the minimum rank represents the best performing algorithm, it can be concluded that gbPSO is found to be the most promising algorithm, which is followed by QL that is also followed by GA*. The naïve algorithm is found as PSO. Moreover, obtained *p*-value (adjusted for ties) for the *best* results strongly suggest the rejection of the null hypotheses, which is based on the equality of medians. Thus, it is statistically demonstrated by the Friedman's test that at least a significantly different pair of algorithms exist.

Table 11
Friedman test ranks for the comparisons with the state-of-the-art.

Algorithms	*Average ranks*
HBPSO/TS	2.90000
GWO_{rbd}	3.20000
gbPSO	3.31666
GWO_{fbd}	3.86666
intAgents	3.93333
Djaya	4.75000
QL	6.26666
GWO	8.26666
DE	8.50000
p-value	0.00000

In order to reveal those significantly different algorithms, pairwise comparisons are performed. Thus, a post-hoc procedure (Nemenyi Test) is applied to avoid Family-Wise-Error-Rate (Derrac et al., 2011). Results, related to these pairwise comparisons are given by Table 10. As one can see from this table, according to the unadjusted *p*-values, all tested algorithms significantly differ from each other for significance level $\alpha = 0.10$. However, more accurate decisions can be made based on the adjusted *p*-values, which are adjusted according to Nemenyi test. In this regard, one can conclude that both QL and gbPSO are found to be significantly better than PSO and GA*, while significant differences do not exist between the pair of algorithms gbPSO vs. QL and GA* vs. PSO.

A similar analysis is carried out for the results of Tables 6–8, where the comparisons with the state-of-the-art are presented. Friedman ranks are given by Table 11. Obtained *p*-values (adjusted for ties) for the best results strongly suggest the rejection of the null hypotheses, which is based on the equality of medians. Moreover, according to the average ranks, one can conclude that HBPSO/TS, GWO_{rbd} and gbPSO are found to be promising algorithms in terms of finding the best solutions. As one can see from the same table, the difference between the ranks increase more dramatically after the first three ranks given to HBPSO/TS, GWO_{rbd} and gbPSO in sequence. This reveals the competitiveness of these optimizers. The naïve algorithm is found as DE.

Pairwise comparisons followed by Nemenyi post-hoc procedure are conducted next. Obtained results are given by Table 12. As one can see from this table, according to the adjusted *p*-values, half of the comparisons point out significant difference for significance level $\alpha = 0.10$. In this regard, one can conclude that the algorithms gbPSO, HBPSO/TS, intAgents, GWO_{rbd} and GWO_{fbd} are found to be significantly better than DE, GWO and QL, while Djaya is found to be significantly better than DE and GWO. Finally QL is found to be significantly better than DE. Enough statistical evidence does not exist to show statistically significant differences between the remaining algorithms.

To sum up, as statistical tests demonstrate gbPSO and QL algorithms are found to be promising algorithms. According to the results

Table 12
Results of pairwise comparisons with the state-of-the-art.

Comparisons	*Adjusted p-values*	*Decision on* H_0	*Comparisons*	*Adjusted p-values*	*Decision on* H_0
HBPSO/TS vs. DE	0.00000	*reject*	*QL vs. GWO*	0.16839	*cannot be rejected*
HBPSO/TS vs. GWO	0.00000	*reject*	*HBPSO/TS vs. Djaya*	0.32000	*cannot be rejected*
GWO_{rbd} *vs. DE*	0.00000	*reject*	*gbPSO vs. HBPSO/TS*	1.00000	*cannot be rejected*
gbPSO vs. DE	0.00000	*reject*	*gbPSO vs.* GWO_{rbd}	1.00000	*cannot be rejected*
GWO_{rbd} *vs. GWO*	0.00000	*reject*	*gbPSO vs.* GWO_{fbd}	1.00000	*cannot be rejected*
gbPSO vs. GWO	0.00000	*reject*	*gbPSO vs. Djaya*	1.00000	*cannot be rejected*
GWO_{fbd} *vs. DE*	0.00000	*reject*	*gbPSO vs. intAgents*	1.00000	*cannot be rejected*
intAgents vs. DE	0.00000	*reject*	*QL vs. Djaya*	1.00000	*cannot be rejected*
GWO_{fbd} *vs. GWO*	0.00000	*reject*	*HBPSO/TS vs.* GWO_{rbd}	1.00000	*cannot be rejected*
intAgents vs. GWO	0.00000	*reject*	*HBPSO/TS vs.* GWO_{fbd}	1.00000	*cannot be rejected*
Djaya vs. DE	0.00000	*reject*	*HBPSO/TS vs. intAgents*	1.00000	*cannot be rejected*
Djaya vs. GWO	0.00002	*reject*	*GWOrbd vs.* GWO_{fbd}	1.00000	*cannot be rejected*
QL vs. HBPSO/TS	0.00006	*reject*	GWO_{rbd} *vs. Djaya*	1.00000	*cannot be rejected*
QL vs. GWO_{rbd}	0.00052	*reject*	GWO_{rbd} *vs. intAgents*	1.00000	*cannot be rejected*
gbPSO vs. QL	0.00108	*reject*	GWO_{fbd} *vs. Djaya*	1.00000	*cannot be rejected*
QL vs. GWO_{fbd}	0.02478	*reject*	GWO_{fbd} *vs. intAgents*	1.00000	*cannot be rejected*
QL vs. intAgents	0.03482	*reject*	*Djaya vs. intAgents*	1.00000	*cannot be rejected*
QL vs. DE	0.05710	*reject*	*DE vs. GWO*	1.00000	*cannot be rejected*

Table 13
Results of the 0–1 KP case study.

Cases		*DE*	*GWO*	*PSO*	*GA**	*QL*	*gbPSO*
off-peak time	*best*	4090	4090	4090	4090	4090	4090
	mean	4090	4090	4090	4090	4090	4090
	hit	30	30	30	30	30	30
peak time	*best*	5760	5760	5760	5760	5760	5760
	mean	5760	5760	5760	5760	5760	5760
	hit	30	30	30	30	30	30
prime time	*best*	9045	9045	9045	9045	9045	9045
	mean	9045	9045	9045	9045	9045	9045
	hit	30	30	30	30	30	30
premium time	*best*	10750	10750	10750	10750	10750	10750
	mean	10750	10750	10750	10750	10750	10750
	hit	30	30	30	30	30	30

of Tables 2–4, although gbPSO exhibits better performance than QL, adjusted *p*-values suggest that a significant difference does not exist between these algorithms. DE, GWO, GA* and PSO with their current forms reported in the present study are found to be naïve algorithms in solving SUKP.

Proposed algorithms can easily be applied to various other binary optimization problems. However, they have several user-defined parameters to be tuned for each different problem. Unfortunately, this is one of the main drawbacks of the proposed algorithms. Nevertheless, putting things together, the algorithms and the procedures proposed in the present study are expected to contribute to the existing literature.

4.4. A real-world implementation

Unfortunately, an exact SUKP related data could not be gathered. Therefore, algorithms are tested on a closely related binary problem, namely, 0–1 KP. All data is burrowed from the related literature (Ntow, 2012).

Ntow (2012) solves an advertisement selection problem and provides data. The aim of this problem is to select a set of advertisements (items) out of all advertisements so as to maximize an objective function that is based on reaching the audience. Therefore, the mentioned problem is modelled as a 0–1 KP.

It is reported that spot advertisements are categorized into four main categories, such as off-peak time, peak time, prime time and premium time, with 21, 18, 23 and 16 advertisement items, respectively. Knapsack capacity is fixed to 360 secs (Ntow, 2012). Algorithms are run for 30 independent replications with *popSize*=20 and *maxIter*=250. Obtained results are presented by Table 13.

Results of Table 13 induce that all tested algorithms achieve same performances. Since the provided instances are relatively smaller-scaled in comparison to the benchmarks used in the present study, this is an expected result. Therefore, it is not possible to come to a conclusion regarding the distinction between the performances of the proposed algorithms for 0–1 KP. Nevertheless, developed algorithms are capable of finding promising results also by using this real-life data.

5. Conclusion

The present paper introduces several algorithms including a machine learning algorithm, namely, Q-learning algorithm for the Set-Union Knapsack Problem (SUKP), which is a natural extension of the well-known 0–1 Knapsack Problem. In the proposed approach, Q-learning algorithm works as a high-level reinforcement method that evaluates the performances of the used optimizers, assigns rewards, promotes or demotes them. Thus, Q-learning algorithm works as an algorithm validation and recommendation system, which selects optimization algorithms as actions at each iteration.

Particle Swarm Optimization (PSO), Genetic Algorithm and a hybrid of these algorithms, namely, genetic-based PSO (gbPSO), which is a new extension of a previously developed algorithm, are used as metaheuristic optimizers in the present study. Moreover, all used algorithms are further modified by adopting various auxiliary procedures such as initial solution generation technique and triggered random immigrants mechanism that is devised to preserve population diversity. Secondarily, a mutation procedure that decreases the introduced diversity is adopted. Thus, more intensified search is encouraged towards the end of search. Moreover, as shown by the present study, the developed gbPSO does not necessarily need transfer functions, which might yield to spatial distance. Finally, all used and enhanced optimizers are employed by the proposed Q-learning mechanism.

The performances of all algorithms are analysed on SUKP, which has a wide range of real-life applications. As demonstrated by the comprehensive experimental study and appropriate statistical tests, promising results are achieved by the proposed algorithms. These findings are additionally validated by the Q-learning algorithm. In this regard, gbPSO and Q-learning algorithm are shown to be promising optimizers for the SUKP.

The developed Q-learning method has some limitations as well. Since it occasionally invokes naïve optimizers, it is not expected to achieve the best results in all instances considering the promising standalone optimization algorithms. However, it brings about two advantages. First, it might help to avoid local optima by allowing some modifications on the current population as similar to mutations in evolutionary algorithms. Secondarily, it provides additional information about the performances of the used algorithms by gathering statistical evidence throughout search. Thus, such information can be used to support statistical findings. Finally, additional CPU burden is required as another limitation.

As shown by the present study, the reported solution methodologies have potential to be used in binary optimization problems. In this regard, applications of the gbPSO to various other binary optimization problems are scheduled as the future work. Moreover, different learning algorithms such as hyper-heuristics can be used as a recommendation system so as to validate the achievements of the used optimization algorithms.

CRediT authorship contribution statement

Fehmi Burcin Ozsoydan: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization, Supervision, Project administration. **İlker Gölcük:** Conceptualization, Methodology, Writing – original draft, Writing – review & editing, Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgement

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

Alpaydin, E., 2021. Machine Learning. MIT Press.

Anghinolfi, D., Paolucci, M., 2009. A new discrete particle swarm optimization approach for the single-machine total weighted tardiness scheduling problem with sequence-dependent setup times. Eur. J. Oper. Res. 193, 73–85. http://dx.doi.org/10.1016/j.ejor.2007.10.044.

Arulselvan, A., 2014. A note on the set union knapsack problem. Discrete Appl. Math. 169, 214–218. http://dx.doi.org/10.1016/j.dam.2013.12.015.

Baykasoglu, A., 2012. Design optimization with chaos embedded great deluge algorithm. Appl. Soft. Comput. 12, 1055–1067. http://dx.doi.org/10.1016/j.asoc.2011.11.018.

Baykasoğlu, A., Ozsoydan, F.B., Senol, M.E., 2020. Weighted superposition attraction algorithm for binary optimization problems. Oper. Res. 20 (4), 2555–2581. http://dx.doi.org/10.1007/s12351-018-0427-9.

Boussaïd, I., Lepagnot, J., Siarry, P., 2013. A survey on optimization metaheuristics. Inf. Sci. 237, 82–117. http://dx.doi.org/10.1016/j.ins.2013.02.041.

Dahmani, I., Ferroum, M., Hifi, M., 2022. Effect of backtracking strategy in population-based approach: the case of the set-union knapsack problem. Cybernet. Syst. 53 (1), 168–185. http://dx.doi.org/10.1080/01969722.2021.2008687.

Derrac, J., García, S., Molina, D., Herrera, F., 2011. A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. Swarm. Evol. Comput. 1, 3–18. http://dx.doi.org/10.1016/j.swevo.2011.02.002.

Diallo, M.H., August, M., Hallman, R., Kline, M., Slayback, S.M., Graves, C., 2017. AutoMigrate: a framework for developing intelligent, self-managing cloud services with maximum availability. Cluster. Comput. 20, 1995–2012. http://dx.doi.org/10.1007/s10586-017-0900-x.

Eberhart, R.C., Kennedy, J., 1995. A new optimizer using particle swarm theory. In: Proceedings of the Sixth IEEE International Symposium on Micro Machine and Human Science, Nagoya, Japan. Piscataway, pp. 39–43. http://dx.doi.org/10.1109/MHS.1995.494215.

Farzaneh-Gord, M., Mohseni-Gharyehsafa, B., Arabkoohsar, A., Ahmadi, M.H., Sheremet, M.A., 2020. Precise prediction of biogas thermodynamic properties by using ANN algorithm. Renew. Energy 147, 179–191. http://dx.doi.org/10.1016/j.renene.2019.08.112.

Gogna, A., Tayal, A., 2013. Metaheuristics: review and application. J. Exp. Theor. Artif. Intell. 25 (4), 503–526. http://dx.doi.org/10.1080/0952813X.2013.782347.

Gölcük, İ., Ozsoydan, F.B., 2020. Evolutionary and adaptive inheritance enhanced grey wolf optimization algorithm for binary domains. Knowl-Based. Syst. 194, 105586. http://dx.doi.org/10.1016/j.knosys.2020.105586.

Gölcük, İ., Ozsoydan, F.B., 2021. Q-learning and hyper-heuristic based algorithm recommendation for changing environments. Eng. Appl. Artif. Intell. 102, 104284. http://dx.doi.org/10.1016/j.engappai.2021.104284.

Goldschmidt, O., Nehme, D., Yu, G., 1994. Note: On the set-union knapsack problem. Nav. Res. Log. 41, 833–842. http://dx.doi.org/10.1002/1520-6750(199410)41:6<833::AID-NAV3220410611>3.0.CO;2-Q.

Gosavi, A., 2015. Simulation-based optimization: An overview. In: Simulation-Based Optimization. In: Operations Research/Computer Science Interfaces Series, Vol. 55, Springer, Boston, MA, http://dx.doi.org/10.1007/978-1-4899-7491-4_3.

Grefenstette, J.J., 1992. Genetic algorithms for changing environments. In Ppsn 2, 137–144.

He, Y., Xie, H., Wong, T.L., Wang, X., 2018. A novel binary artificial bee colony algorithm for the set-union knapsack problem. Future Gener. Comput. Syst. 78, 77–86. http://dx.doi.org/10.1016/j.future.2017.05.044.

Holland, J.H., 1975. Adaptation in Natural and Artificial Systems. The University of Michigan Press, Ann Arbor, MI.

Hussain, K., Mohd Salleh, M.N., Cheng, S., Shi, Y., 2019. Metaheuristic research: a comprehensive survey. Artif. Intell. Rev. 52 (4), 2191–2233. http://dx.doi.org/10.1007/s10462-017-9605-z.

Jagiello, J., Taylor, R., 2017. On the set-union budget scenario problem. In; proceedings of the 22nd International Congress on Modelling and Simulation, Hobart, Tasmania, Australia, December, pp. 653-659.

Jang, J.-S.R., Sun, C.-T., Mizutani, E., 1997. Neuro-Fuzzy and Soft Computing - a Computational Approach to Learning and Machine Intelligence. Prentice Hall, Upper Saddle River, New Jersey.

Kellerer, H., Pferschy, U., Pisinger, D., 2004. Knapsack Problems. Springer, Berlin.

Kennedy, J., Eberhart, R., 1995. Particle swarm optimization. In: Proceedings of the IEEE International Conference on Neural Network, Perth, Australia, pp. 1942-1948.

Khan, J., Wei, J.S., Ringner, M., Saal, L.H., Ladanyi, M., Westermann, F., et al., 2001. Classification and diagnostic prediction of cancers using gene expression profiling and artificial neural networks. Nat. Med. 7 (6), 673–679. http://dx.doi.org/10.1038/89044.

Kuo, R.J., Zulvia, F.E., Suryadi, K., 2012. Hybrid particle swarm optimization with genetic algorithm for solving capacitated vehicle routing problem with fuzzy demand-a case study on garbage collection system. Appl. Math. Comput. 219, 2574–2588. http://dx.doi.org/10.1016/j.amc.2012.08.092.

Lian, Z., Gu, X., Jiao, B., 2006. A similar particle swarm optimization algorithm for permutation flowshop scheduling to minimize makespan. Appl. Math. Comput. 175, 773–785. http://dx.doi.org/10.1016/j.amc.2005.07.042.

Lin, G., Guan, J., Li, Z., Feng, H., 2019. A hybrid binary particle swarm optimization with tabu search for the set-union knapsack problem. Expert Syst. Appl. 135, 201–211. http://dx.doi.org/10.1016/j.eswa.2019.06.007.

Lister, W., Laycock, R.G., Day, A.M., 2010. A dynamic cache for real-time crowd rendering. In: Computer Graphics Forum.

Maleki, B., Ghazvini, M., Ahmadi, M.H., Maddah, H., Shamshirband, S., 2019. Moisture estimation in cabinet dryers with thin-layer relationships using a genetic algorithm and neural network. Mathematics 7 (11), 1042. http://dx.doi.org/10.3390/math7111042.

Marinakis, Y., Marinaki, M., 2010. A hybrid genetic–particle swarm optimization algorithm for the vehicle routing problem. Expert Syst. Appl. 37, 1446–1455. http://dx.doi.org/10.1016/j.eswa.2009.06.085.

Marinakis, Y., Marinaki, M., Dounias, G., 2010. A hybrid particle swarm optimization algorithm for the vehicle routing problem. Eng. Appl. Artif. Intell. 23, 463–472. http://dx.doi.org/10.1016/j.engappai.2010.02.002.

Mirjalili, S., Hashim, S.Z.M., Sardroudi, H.M., 2012. Training feedforward neural networks using hybrid particle swarm optimization and gravitational search algorithm. Appl. Math. Comput. 218 (22), 11125–11137. http://dx.doi.org/10.1016/j.amc.2012.04.069.

Mirjalili, S., Mirjalili, S.M., Lewis, A., 2014. Grey wolf optimizer. Adv. Eng. Softw. 69, 46–61. http://dx.doi.org/10.1016/j.advengsoft.2013.12.007.

Mosadegh, H., Fatemi Ghomi, S.M.T., Süer, G.A., 2020. Stochastic mixed-model assembly line sequencing problem: Mathematical modeling and Q-learning based simulated annealing hyper-heuristics. Eur. J. Oper. Res. 282, 530–544. http://dx.doi.org/10.1016/j.ejor.2019.09.021.

Ntow, A.S., 2012. Knapsack problem a case study of metro t.v. a television station in Ghana (MSc Thesis).

Ozsoydan, F.B., 2019. Artificial search agents with cognitive intelligence for binary optimization problems. Comput. Ind. Eng. 136, 18–30. http://dx.doi.org/10.1016/j.cie.2019.07.007.

Ozsoydan, F.B., Baykasoglu, A., 2019. A swarm intelligence-based algorithm for the set-union knapsack problem. Future Gener. Comput. Syst. 93, 560–569. http://dx.doi.org/10.1016/j.future.2018.08.002.

Ozsoydan, F.B., Sipahioglu, A., 2013. Heuristic solution approaches for the cumulative capacitated vehicle routing problem. Optimization 62, 1321–1340. http://dx.doi.org/10.1080/02331934.2013.841158.

Pan, Q.K., Tasgetiren, M.F., Liang, Y.C., 2008. A discrete particle swarm optimization algorithm for the no-wait flowshop scheduling problem. Comput. Oper. Res. 35, 2807–2839. http://dx.doi.org/10.1016/j.cor.2006.12.030.

Riondato, M., Vandin, F., 2014. Finding the true frequent itemsets. In: proceedings of the 2014 SIAM International Conference on Data Mining, pp. 497-505.

Storn, R., Price, K., 1995. Differential evolution-a simple and efficient adaptive scheme for global optimization over continuous spaces. Technical Report TR-95-012, ICSI, (Available via ftp from https://ftp.icsi.berkeley.edu/pub/techreports/1995/tr-95-012.ps.Z).

Su, L., Zhou, Y., 2016. Tolerating correlated failures in massively parallel stream processing engines. In: Proceedings of IEEE Conference in Data Engineering (ICDE), 517-528, Helsinki, Finland.

Taylor, R., 2016. Approximations of the densest k-subhypergraph and set union knapsack problems. arXiv preprint arXiv:1610.04935.

Tseng, C.T., Liao, C.J., 2008. A discrete particle swarm optimization for lot-streaming flowshop scheduling problem. Eur. J. Oper. Res. 191, 360–373. http://dx.doi.org/10.1016/j.ejor.2007.08.030.

Turkoglu, B., Kaya, E., 2020. Training multi-layer perceptron with artificial algae algorithm. Eng. Sci. Tech. Int. J. 23 (6), 1342–1350. http://dx.doi.org/10.1016/j.jestch.2020.07.001.

Uslu, S., Celik, M.B., 2018. Prediction of engine emissions and performance with artificial neural networks in a single cylinder diesel engine using diethyl ether. Eng. Sci. Tech. Int. J. 21 (6), 1194–1201. http://dx.doi.org/10.1016/j.jestch.2018.08.017.

Wang, H., Wang, D., Yang, S., 2009. A memetic algorithm with adaptive hill climbing strategy for dynamic optimization problems. Soft. Comput. 13, 763–780. http://dx.doi.org/10.1007/s00500-008-0347-3.

Watkins, C.J.C.H., Dayan, P., 1992. Q-learning. Mach. Learn. 8, 279–292. http://dx.doi.org/10.1007/BF00992698.

Wei, Z., Hao, J.K., 2019. Iterated two-phase local search for the set-union knapsack problem. Future Gener. Comput. Syst. 101, 1005–1017. http://dx.doi.org/10.1016/j.future.2019.07.062.

Wei, Z., Hao, J.K., 2021a. Multistart solution-based tabu search for the set-union knapsack problem. Appl. Soft. Comput. 105, 107260. http://dx.doi.org/10.1016/j.asoc.2021.107260.

Wei, Z., Hao, J.K., 2021b. Kernel based tabu search for the set-union knapsack problem. Expert. Syst. Appl. 165, 113802. http://dx.doi.org/10.1016/j.eswa.2020.113802.

Wu, C., He, Y., 2020. Solving the set-union knapsack problem by a novel hybrid jaya algorithm. Soft. Comput. 24 (3), 1883–1902. http://dx.doi.org/10.1007/s00500-019-04021-3.

Xia, W., Wu, Z., 2005. An effective hybrid optimization approach for multi-objective flexible job-shop scheduling problems. Comput. Ind. Eng. 48, 409–425. http://dx.doi.org/10.1016/j.cie.2005.01.018.

Yilmaz, M., Kayabasi, E., Akbaba, M., 2019. Determination of the effects of operating conditions on the output power of the inverter and the power quality using an artificial neural network. Eng. Sci. Tech. Int. J. 22 (4), 1068–1076. http://dx.doi.org/10.1016/j.jestch.2019.02.006.